Part 2

① Shift Left: Multiplication by 2^i

sll → shift left logical

0000 1100 —sll→ 000 11000 —sll→ 00 110000

12 — $12\times2^1 = 24$ — $12\times2^2 = 48$

* a = b × 8

slli x5, x6, 3

a = x5
b = x6

a = b × 16

sll x5, x6, 4

What happens if we shift a value more than 63 time?

Ans: Will become zero

a = b × 4

slli x5, x6, 2

a = b × 5
= b × (4 + 1)
= b × 4 + b

slli x5, x6, 2 // x5 = b × 2²
x5 = b × 4

add x7, x5, x6 // x7 = x5 + x6
↓
a = b × 4 + b

a = x7
b = x6

Represent int n = a * 19 in assembly code.

n = a × (16 + 2 + 1)
= 16a + 2a + a

slli x5, x22, 4
slli x6, x22, 1
add x21, x5, x6
add x21, x21, x22

n = x21
a = x22
19 = 16 + 2 + 1

② Shift Right: Division by 2^i

0011 000 —srl→ 0011 00 —srl→ 000011 0

48 — 24 — 12

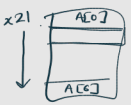
Convert A[3] = A[6] + 1 into assembly code

A is doublewordded array

$Base_A$ = x21

ld x5, 48[x21] // x5 = A[6]
(offset)
addi x6, x5, 1 // x6 = A[6] + 1

sd x6, 24[x21] // A[3] = A[6] + 1



6 × 8
= 48

Convert A[3] = A[g] + 1 into assembly code

```
slli  x5, x25, 3     // x5 = g*8
add   x6, x21, x5    // x6 = A[0] + g*8
ld    x7, 0[x6]      // x7 = A[0] + g*8 + 0
addi  x28, x7, 1     // x28 = A[6] + 1
sd    x28, 24[x21]   // A[3] = A[6] + 1
```

A is doubleworded array

Base_A = x21

g = x25

Convert A[3] = A[B[8]] + 1 into assembly code

```
ld    x5, 64[x21]    // x5 = B[8]
slli  x5, x5, 3      // x5 = B[8] * 8
add   x6, x20, x5    // x6 = A + B[8]*8
ld    x6, 0[x6]
addi  x6, x6, 1      // x6 = A[B[8]] + 1
sd    x6, 24[x20]    // A[3] = A[B[8]] + 1
```

Base_A = x20

Base_B = x21

15. Translate the following assembly to equivalent C code.
Assume : A base in x21, B base in x20, i in x22

```
slli x10, x22, 3
add  x10, x10, x20
ld   x5, 0(x10)
addi x5, x5, 1
sd   x5, 24(x21)
```

Ans:

x10 = i*8

x10 = Base_B + i*8

x5 = B[i]

x5 = B[i] + 1

A[3] = B[i] + 1

24/8 = 3

Ans: A[3] = B[i] + 1

12. Translate the following assembly to equivalent C code.

```
slli x30, x5, 3
add x30, x10, x30
slli x31, x6, 3
add x31, x11, x31
ld x5, 0(x30)
addi x12, x30, 8
ld x30, 0(x12)
add x30, x30, x5
sd x30, 0(x31)
```

Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7, x28, and x29, respectively. Assume that the base address of the

Arrays A and B are in registers x10 and x11, respectively.

24 (x20)

x20+3*8

```
slli x30, x5, 3          // x30 = f*8
add x30, x10, x30        // x30 = Base_A + f*8
slli x31, x6, 3          // x31 = g*8
add x31, x11, x31        // x31 = Base_B + g*8
ld x5, 0(x30)            // x5 = A[f]
addi x12, x30, 8         // x12 = (Base_A + f*8) + 8
ld x30, 0(x12)           // x30 = A[f+1]
add x30, x30, x5         // x30 = A[f+1] + A[f]
sd x30, 0(x31)           // B[g] = A[f+1] + A[f]
```

③ AND : Bit Masking

$C = A \cdot B$

and x9, x10, x11

$C = x9$
$A = x10$
$B = x11$

A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

0 and s^{th} = 0 (mask)

1 and s^{th} = same

x10 = 1001
x11 = 1101
x9 = 1001
} Bitwise operation

Given x10 = 0000 . . . . . 0000 1100 1100. Keep 7-4 and 2-1 the same. Make everything else zero.

x11 = 0000 . . . . 00001111 0110

and x9, x10, x11

④ OR : Include Bits

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

0 on sth = same

1 on sth = 1

or x9, x10, x11

x10 = 0000 . . . . . 0000 1100 0000. Include the highlighted part/make 1

x11 = 0000 . . . 000 0011 0000

OR x9, x10, x11

⑤ XOR : Can work as Buffer and Not

x9 = 111 ← flip
x10 = 001

xor x11, x9, x10

A	B	A ⊕ B	
0	0	0	→ Buffer
0	1	1	→ NOT
1	0	1	→ Buffer
1	1	0	→ NOT